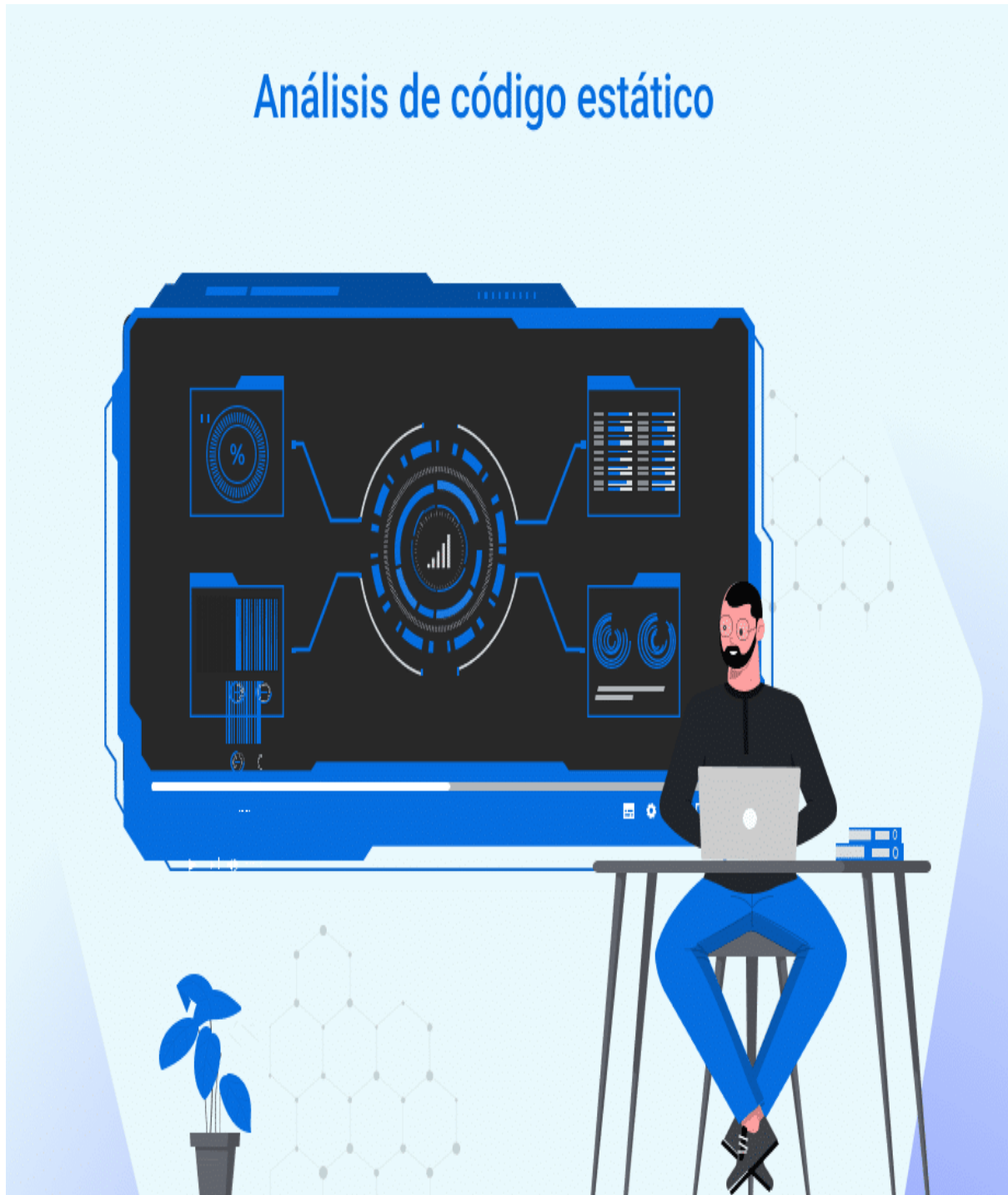


Práctica 4.2 Analizadores de Código (CE4.c & CE4.d & CE4.e)



Nombre: Mateo Daniel Ortuño Ovando
Asignatura: Entornos de desarrollo

| fecha de inicialización: 18/02/2026
| fecha de finalización: 22/02/2026

ndice del Proyecto

1. SonarQube

- 1.1. Instalación
- 1.2. Plugin de NetBeans

2. Configurando el analizador

3. Trabajando con SonarQube

4. Patrones de refactorización con NetBeans

5. Qué hacer

- 5.2. Corrección del proyecto y documentación (Regla S109: Magic Numbers)
 - *Explicación detallada de la regla S109 y corrección del código siguiendo las indicaciones de SonarQube.*
- 5.3. Uso de patrones de refactorización en NetBeans
 - *Aplicación de sugerencias del analizador para mejorar la legibilidad del código.*
- 5.4. Evidencias de corrección (Capturas de pantalla)
 - *Muestra del estado final del proyecto tras aplicar las mejoras.*

1. SonarQube.

_ SonarQube es una plataforma de análisis estático de código abierto diseñada para evaluar y mejorar la calidad del código fuente en proyectos de desarrollo de software. Proporciona una amplia gama de herramientas y funcionalidades que permiten a los equipos de desarrollo:

- **Análisis Automático de Código:** Para identificar problemas de calidad, incluyendo errores, bugs potenciales, vulnerabilidades de seguridad, y violaciones de estándares de codificación.
- **Detección de Problemas de Seguridad:** Como vulnerabilidades conocidas y prácticas de codificación inseguras.
- **Medición de la Deuda Técnica:** Calcula y visualiza la deuda técnica asociada con el código fuente, que incluye el costo estimado de corregir problemas y mejorar la calidad del código.
- **Gestión de Código Duplicado:** Ayuda a reducir la complejidad y mejorar la mantenibilidad del código.
- **Integración con Herramientas de Desarrollo:** Permite ejecutar análisis de código automáticamente como parte del proceso de desarrollo y proporcionar retroalimentación directamente dentro del entorno de desarrollo integrado (IDE).
- **Informes y Visualizaciones:** Ofrece informes detallados que muestran la calidad del código en diferentes áreas, permitiendo a los equipos de desarrollo comprender mejor los problemas y tomar medidas correctivas.

1.1. Instalación

_ Para descargar SonarQube nos dirigimos al sitio web oficial <https://www.sonarqube.org/downloads>.

_ Una vez que se haya completado la descarga, descomprimos el archivo ZIP en un directorio adecuado para almacenar programas y ejecutables (por ejemplo, "C:\Program Files").

_ Lo guardamos correctamente, en archivos de programa el nombre del archivo extraído se vería así:



sonarqube-2026.1.0.119033

_Ya tenemos lista para funcionar la herramienta.

_Nos vamos a la línea de comandos, y llamamos al script “StartSonar.bat”. Por ejemplo, en Windows escribimos: C:\Program Files\sonarqube-10.4.1.88267\bin\windows-x86-64\StartSonar.bat 2 Se empezará a lanzar el servicio, el cual estará operativo cuando aparezca el mensaje “SonarQube is operational”

```
C:\Users\kaexd>cd C:\Program Files\sonarqube-2026.1.0.119033\bin\windows-x86-64

C:\Program Files\sonarqube-2026.1.0.119033\bin\windows-x86-64>StartSonar.bat
Starting SonarQube...
2026.02.20 11:01:17 INFO app[][o.s.a.AppFileSystem] Cleaning or creating temp directory C:\Program Files\sonarqube-2026.1.0.119033\temp
2026.02.20 11:01:17 INFO app[][o.s.a.es.EsSettings] Elasticsearch listening on [HTTP: 127.0.0.1:9001, TCP: 127.0.0.1:{}]
2026.02.20 11:01:17 INFO app[][o.s.a.ProcessLauncherImpl] Launch process[ELASTICSEARCH] from [C:\Program Files\sonarqube-2026.1.0.119033\elasticsearch]: C:\Program Files\Java\jdk-25\bin\java -Xms4m -Xmx64m -XX:+UseSerialGC -Dcli.name=server -Dcli.script=./bin/elasticsearch -Dcli.libs=lib/tools/server-cli -Des.path.home=C:\Program Files\sonarqube-2026.1.0.119033\elasticsearch -Des.path.conf=C:\Program Files\sonarqube-2026.1.0.119033\data/es8\config -Des.distribution.type=tar -cp C:\Program Files\sonarqube-2026.1.0.119033\elasticsearch\lib\*;C:\Program Files\sonarqube-2026.1.0.119033\elasticsearch\lib\cli-launcher\* org.elasticsearch.launcher.CliToolLauncher
2026.02.20 11:01:17 INFO app[][o.s.a.SchedulerImpl] Waiting for Elasticsearch to be up and running
Feb 20, 2026 11:01:22 A.M. sun.util.locale.provider.LocaleProviderAdapter <clinit>
INFO: Invalid locale provider adapter "COMPAT" ignored.
Feb 20, 2026 11:01:22 A.M. sun.util.locale.provider.LocaleProviderAdapter <clinit>
WARNING: COMPAT locale provider has been removed
```

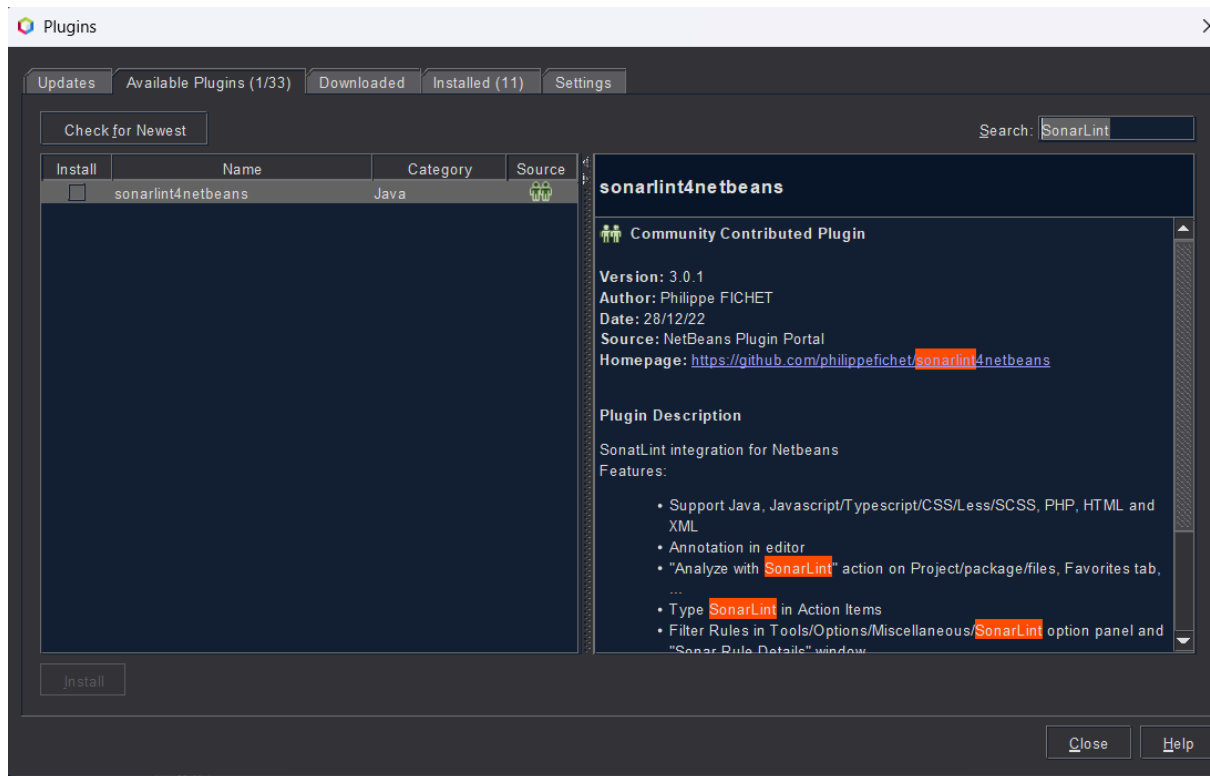
_Lo probamos en nuestro cmd, primero vamos a la ubicación exacta del archivo con cd. Y la ubicación, y luego ejecutamos el archivo .bat

```
INFO app[][o.s.a.SchedulerImpl] Process[ce] is up 2026.02.20 11:02:53 INFO
app[][o.s.a.SchedulerImpl] SonarQube is operational
```

1.2. Plugin de NetBeans

_NetBeans proporciona una integración directa de SonarQube en el entorno de desarrollo integrado a través de un plugin.

_Para ello, simplemente nos dirigimos al menú "Tools" y seleccionamos "Plugins".



_Marcamos la casilla de instalar, he instalamos el plugin seleccionado.

2. Configurando el analizador

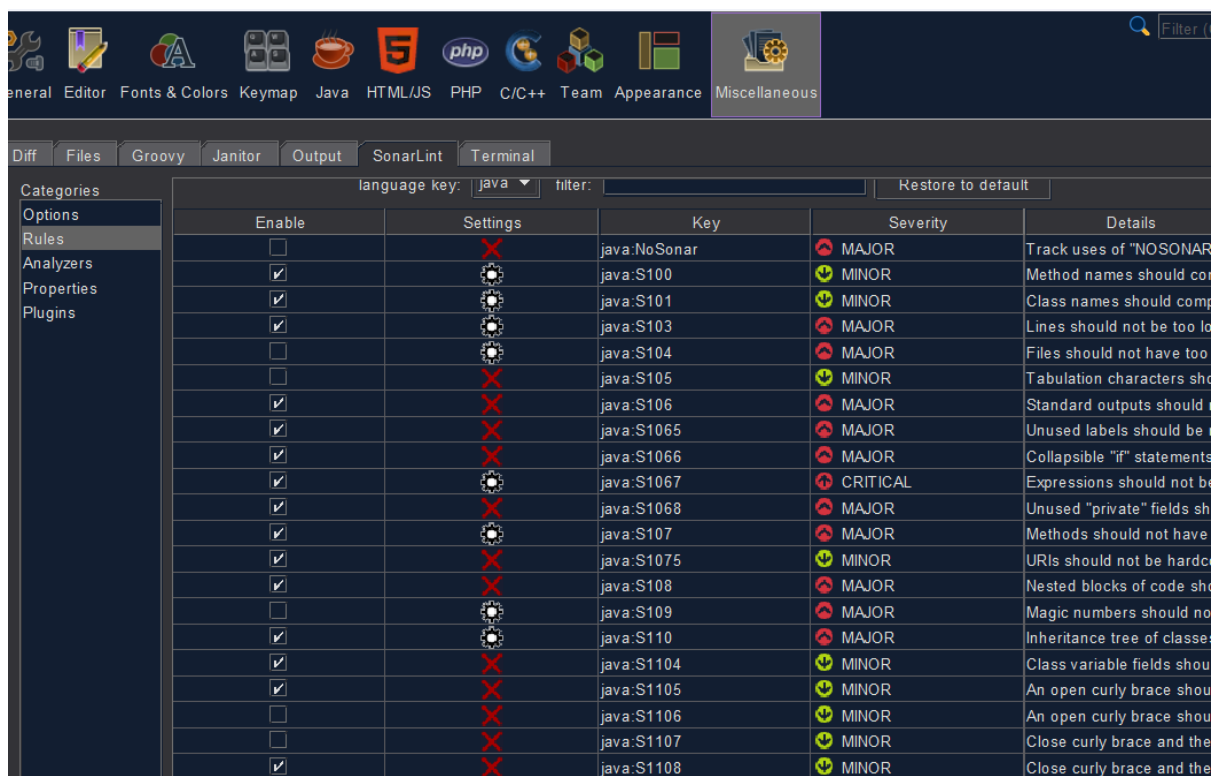
_Vamos a configurar ahora nuestro analizador. Pinchamos en Tools->Options, y se abrirá una ventana para realizar la configuración. Pinchamos en la opción

“Miscellaneous”, y se abrirá la configuración de todos los plugin. Pinchamos en la pestaña “SonarLint”

_Vamos a garantizar, que entre las reglas de análisis de java se encuentren marcadas las siguientes:

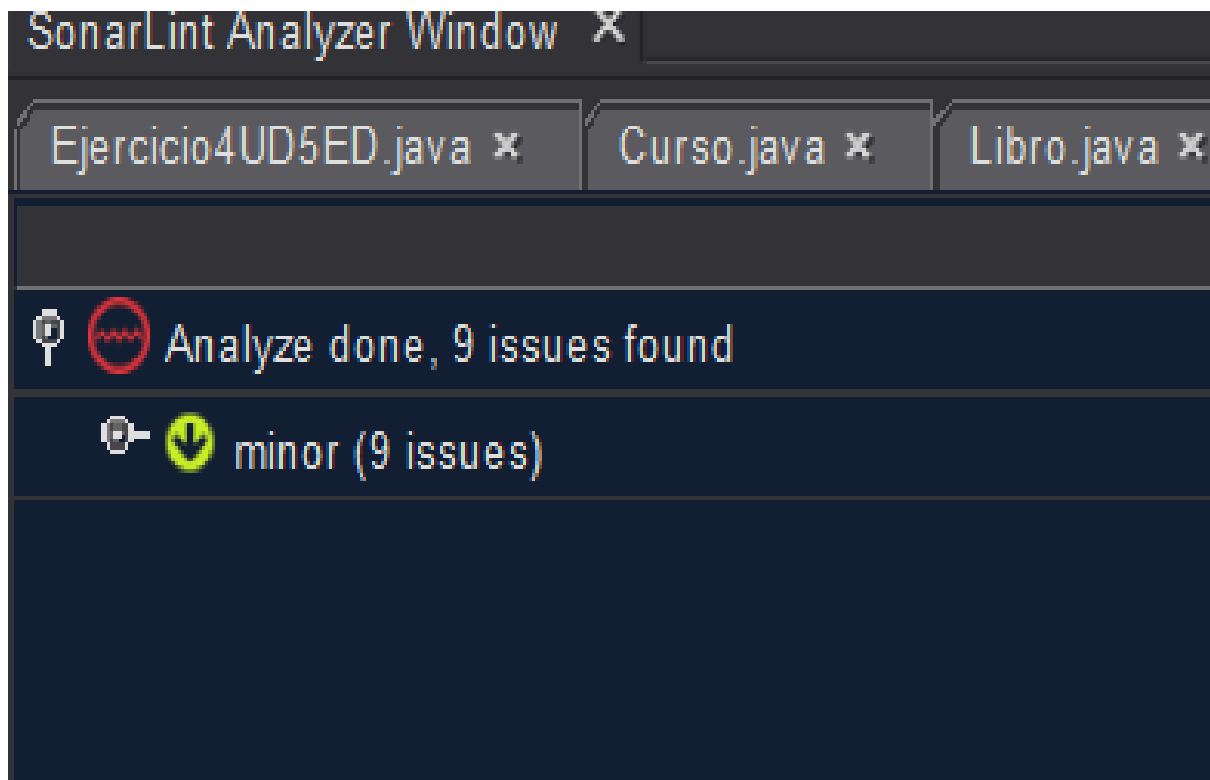
- S103-Lines should not be too long.
- S1067-Expressions should not be too complex.
- S1105- An open curly brace should be located at the end of a line. • S1108- Close curly brace and the next “else”, “catch” and “finally” keywords should be on two different lines.
- S1109- A close curly brace should be located at the beginning of a line.
- S121- Control structures should use curly braces. Y quitamos las siguientes opciones:
- S106-Standard outputs should not be used directly to log anything

_En efecto en nuestro Ide esas reglas estan marcadas y si no las marcamos y aplicamos cambios.



3. Trabajando con SonarQube

_Vamos a analizar el proyecto disponible en Moodle mediante SonarLint. Para ello, pinchamos en nuestro proyecto y nos dirigimos a “Tools->Analyze with SolarLint”. Comenzará a realizar el análisis:



_Solo nos queda aplicar el plugin con sus sugerencias y corregir fichero por fichero de nuestro proyecto java.

4. Patrones de refactorización con NetBeans

_ NetBeans suele indicarnos aquellas expresiones que pueden ser refactorizadas mediante un símbolo de advertencia al lado de la expresión susceptible de cambio.



_Si pinchamos en el símbolo, o pulsamos Alt+Enter, nos dará una sugerencia de refactorización, que en muchos casos será adecuada.

5. Qué hacer

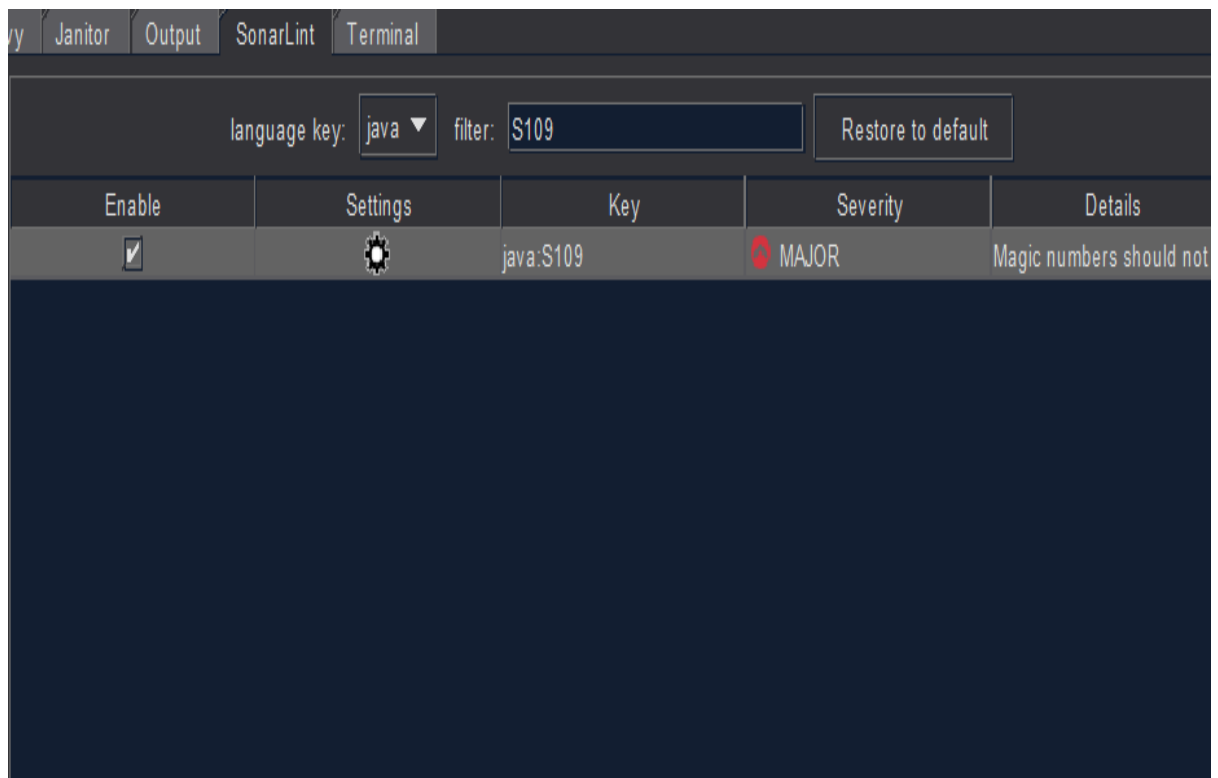
- ☐ 1. Documento con capturas el proceso de instalación de SonarQube y la instalación del plugin en NetBeans.

- ☐ 2. Corrige el proyecto, siguiendo lo que te indica SonarQube, y documéntalo en un archivo PDF. Indica qué son los “Magic Numbers” de los que habla la regla S109.
- ☐ 3. En el mismo proyecto, utiliza las sugerencias de patrones de refactorización que te ofrece el analizador de NetBeans para hacer más legible el código, y documéntalas.
- ☐ 4. Por último realiza capturas indicando que se ha corregido del proyecto.

5.2 Corrige el proyecto, siguiendo lo que te indica SonarQube, y documéntalo en un archivo PDF. Indica qué son los “Magic Numbers” de los que habla la regla S109.

_Con respecto a este objetivo de la practica, nos vamos devuelta a las reglas que nos pedian que verifiquemos que estaban habilitadas en subtitulo 2 de la practica.

_Entre ellos no se encuentra el S109, por lo que al analizar los ficheros, no nos va a saltar esta regla para analizar el codigo, por lo que buscamos la llave de la regla y en detalles vemos magic numbers, no estaba marcada por lo que la marcamos y ahora si nos saltara la regla de “magic numbers”.



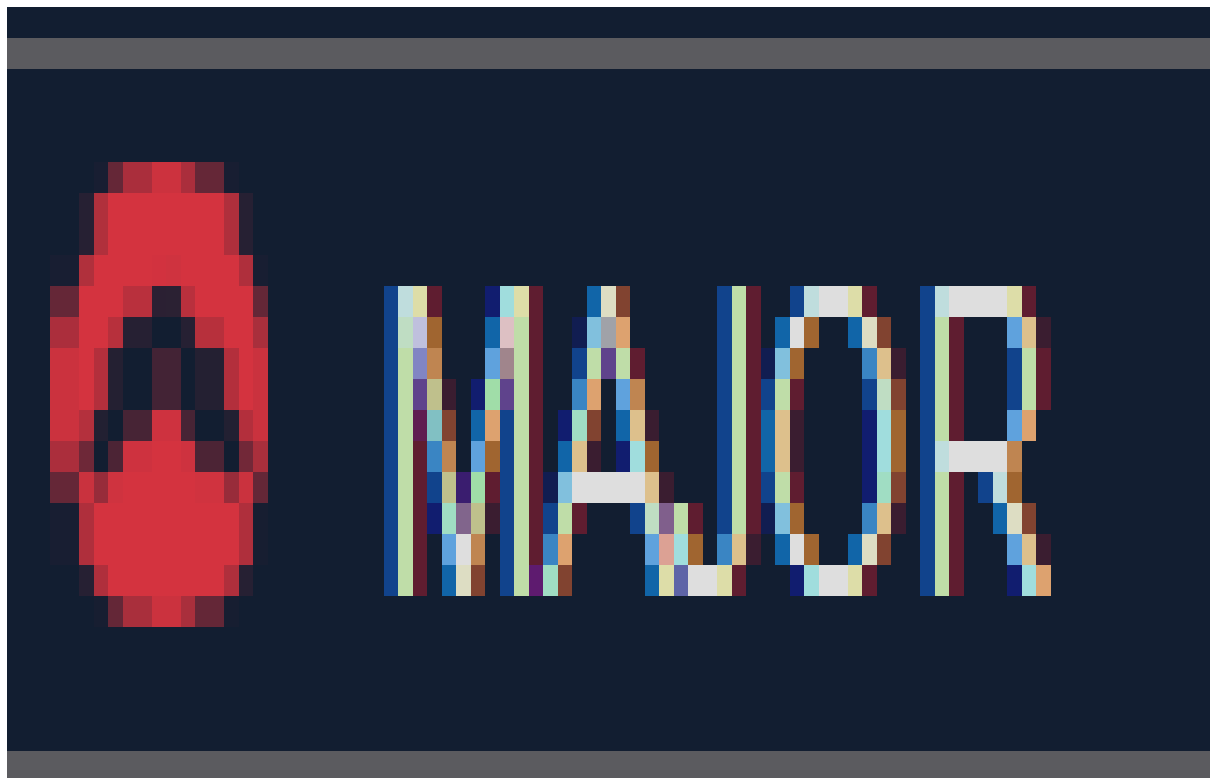
_Los magic numbers, hace referencia a que un numero por ejemplo 86400 es menos claro que usar una constante SECONDS_IN_A_DAY;

_A continuacion enseñare capturas de las sugerencias de mejora gracias al analizador de sonnarqube, primero debemos saber que significa severity que serian el simbolo y seguido de las palabras:

- ★ Critical (Crítico): Errores con baja probabilidad de impactar en producción o problemas que representan una falla de seguridad. Deben ser revisados lo antes posible. Ejemplos: bloques catch vacíos o vulnerabilidades de SQL injection.



- ★ Major (Mayor): Defectos de calidad que pueden impactar significativamente en la productividad del desarrollador. Ejemplos: bloques de código duplicados, parámetros sin usar o falta de cobertura de pruebas.



- ★ Minor (Menor): Defectos de calidad con un impacto ligero en la productividad. Ejemplos: líneas de código demasiado largas o sentencias "switch" con menos de 3 casos.

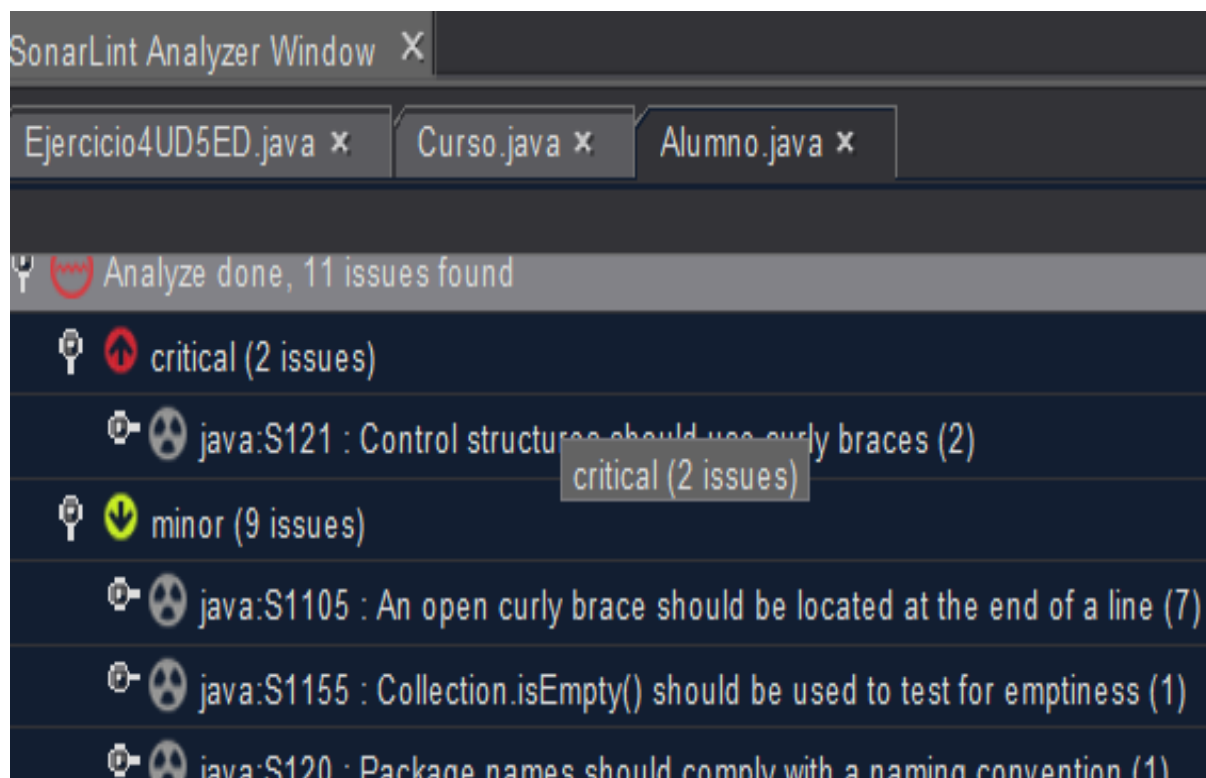


- ★ Info (Informativo): No representan ni un error ni un defecto de calidad; son simplemente hallazgos o recordatorios sin impacto esperado en la aplicación.

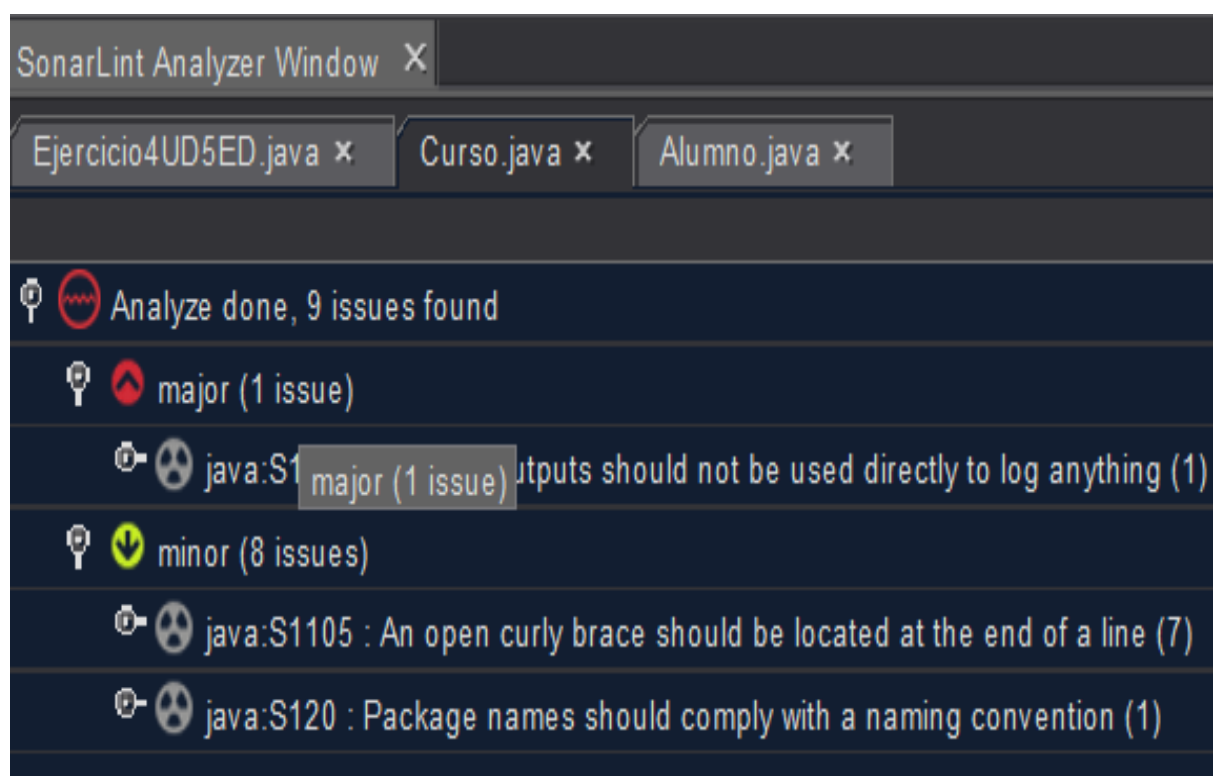


_Una vez explicado cada grado de importancia de cada simbolo y signo, vamos a ver nuestros ficheros java:

Alumno:



Curso:



Ejercicio4UD5ED:

ejercicio4_ud5_ed.Ejercicio4UD5ED >

SonarLint Analyzer Window X

Ejercicio4UD5ED.java X Curso.java X Alumno.java X

  Analyze done, 6 issues found

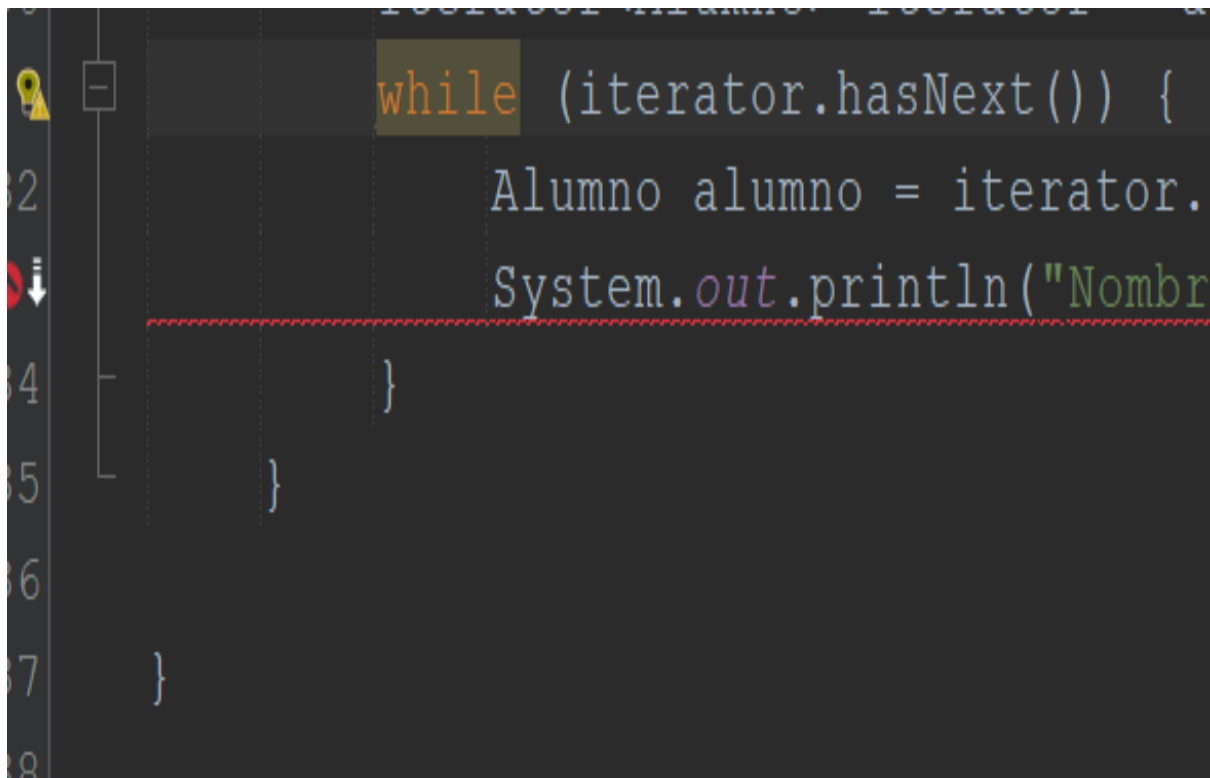
  major (5 issues)

-   java:S103 : Lines should not be too long (1)
-   java:S106 : Standard outputs should not be used directly to log anything (2)
-   java:S109 : Magic numbers should not be used (2)

  minor (1 issue)

5.3 En el mismo proyecto, utiliza las sugerencias de patrones de refactorización que te ofrece el analizador de NetBeans para hacer más legible el código, y documéntalas.

_Analizamos los tres ficheros y verificamos las líneas en donde al lado izquierdo aparezca el símbolo de la bombilla, el cual es el símbolo de sugerencias del analizador de netbeans, en este caso lo encontramos en el fichero Ejercicio4UD5ED Línea 13.



5.4. Por último realiza capturas indicando que se ha corregido del proyecto.

_Estas son las correcciones del proyecto:

Empezamos con lo errores del fichero

Alumno

```
if (cantidadAlumnos > 0)
{
    return (double) sumaEdades / cantidadAlumnos;
}
else
{
    return 0;
}
```

_el error S121 critical "*Control structures should use curly braces*.", Es critico ya que las llaves en los if y else son el principal y basico problema en compilacion, ya sean de una linea, de varias ni se diga.

```

public Alumno(String nombre, int edad, String codigo){
    this.nombre = nombre;
    this.edad = edad;
    this.codigo = codigo;
}

public String getNombre(){
    return nombre;
}

public int getEdad(){
    return edad;
}

public String getCodigo(){
    return codigo;
}

public void setNombre(String nombre){
    this.nombre = nombre;
}

```

_java:S1105: "An open curly brace should be located at the end of a line",
 básicamente este error se basa en que el inicio de llave debe estar al inicio de la
 línea, no como estaba antes, este estaba mayormente en los getters, setters y if
 else.

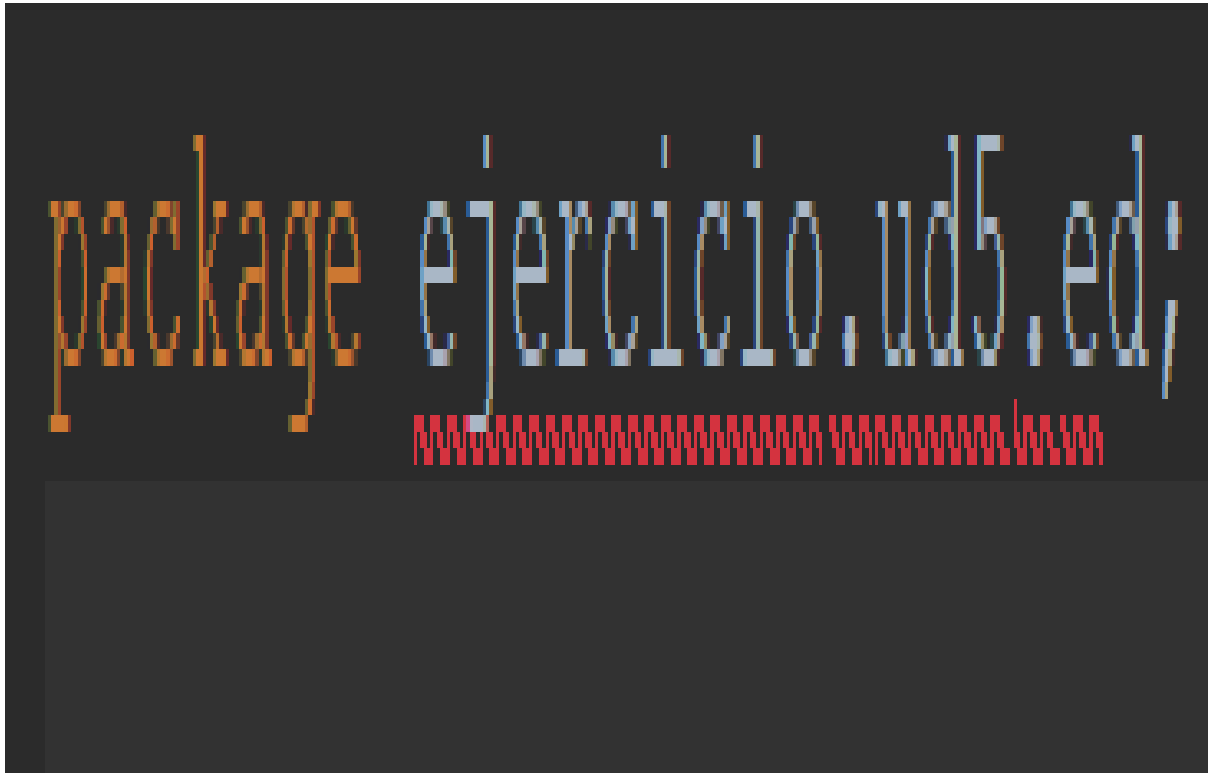
```

// FUNCION para calcular la
public static double calcula
    if (alumnos.isEmpty()) {
        return 0;
    }
    Iterator<Alumno> iterate

```

_El error java:S1155: "Collection.isEmpty() should be used to test for emptiness"

Se basa en que Aunque funciona, usar `.isEmpty()` es más legible y, en algunos tipos de listas muy grandes o complejas, es más eficiente que contar todos los elementos (`size`) solo para saber si hay cero.



_El siguiente error java:S120: "Package names should comply with a naming convention", Basicamente se debe a que los nombres por convenio de java son mejores para los package, para ello tuve que cambiar con refactor el nombre del paquete donde se encontraban los archivos.

Curso

```
56 public void mostrarEdad
    double edadPromedio
    System.out.println(
59 }
60 }
61
```

_java:S106: Standard outputs should not be used directly to log anything (Major)

Este error es un poco complejo de arreglar, ya que el error consiste en que en un programa profesional el System.out esta mal aplicado o no se usa porque no sabemos cuando o donde aparecera en pantalla, para ello se arregla con el conocimiento de logs, Todavia no lo hemos mirado, pero sirve el warning para saber que es y porque esta.

_Luego teniamos el mismo error que Alumno java:S1105: "An open curly brace should be located at the end of a line, como ya nos sabemos este lo corregimos sin mas.

Ejercicio4UD5ED

```

System.out.println("Lista de alumnos");
List<Alumno> alumnos = curso.getAlumnos();
Iterator<Alumno> iterator = alumnos.iterator();
while (iterator.hasNext()) {
    Alumno alumno = iterator.next();
    System.out.println("Nombre: " + alumno.getNombre());
}

```

_java:S106: Standard outputs should not be used directly (2 avisos)

Como en el código anterior, Este tiene dos System.out, ya sabemos la solución pero no como aplicarla debido a su complejidad.

```

private static final int EDAD_MINIMA_JUAN = 20;
private static final int EDAD_MINIMA_MARIA = 21;

public static void main(String[] args) {
    Curso curso = new Curso("Matemáticas");
    Alumno alumno1 = new Alumno("Juan", EDAD_MINIMA_JUAN, "A001");
    Alumno alumno2 = new Alumno("María", EDAD_MINIMA_MARIA, "A002");
}

```

_java:S109: Magic numbers should not be used (2 avisos), En estos errores como explicamos en el punto 5.2, son por la falta de especificación en números, para ello

lo arreglamos fuera del main con una constante estatica, asi el valor no cambia y el error ya no salta.

```
Alumno alumno = Iterator.next();  
  
System.out.println("Nombre: " + alumno.getNombre() +  
    ", Edad: " + alumno.getEdad() +  
    ", Código: " + alumno.getCodigo());  
  
}
```

_java:S103: Lines should not be too long (1 aviso), Este error es facil de definir, la linea es muy larga, para ello solo hacemos saltos de linea en medio.